JuliarmaResize

Tejahni Desire,[1]
[1]Wake Forest University, 1834 Wake Forest Road, Winston-Salem, NC 27109, USA

May 31st - August 9th

ABSTRACT

"Lets, uh, resize some stuff"
-John Resizer

1 INTRODUCTION

The fundamental objective for `JuliarmaResize` (or `Juliarma`) is the transformation of a given grid $\mathcal{G}_0$ with a specific resolution $N_{old} = N_{0,x1} \times N_{0,x2} \times N_{0,x3}$ to a new grid $\mathcal{G}_{\rm new}$ with a different resolution $N_{\rm new} = N_{x1} \times N_{x2} \times N_{x3}$. When changing grids, we must also change the fields associated with that grid. For `KHARMA`, the variables we must change are the primitives $\rho_{\rm prim}$, $u_{\rm prim}$, $\mathbf{u}_{\rm prim}$, and the conservatives $\rho_{\rm con}$,$u_{\rm con}$, $\mathbf{u}_{\rm con}$, $\mathbf{B}_{\rm con,f}$.

1.1 Face and Cell Centering

1.1.1 Indices

There are two different field representations for fields in `KHARMA`, face-centered fields and cell-centered fields. The latter means that the value at that the matrix values are indexed at positions representing the center of the cell, meanwhile face values are indexed at faces. The indices here are not trivial, so we will take the time to carefully define them.

A pointwise face-centered matrix vector F has components F^1, F^2, F^3. Here, pointwise indicates that the value of $F^{\rm d}$ at a given point is the field value normal to direction $\hat{x}_{\rm d}$, evaluated exactly at that face's coordinates.

For a cell whose center is located at the coordinates $\vec{x} = (x, y, z)$, the F^1 component has faces of interest for that cell. The face coordinates are $\vec{x}_{\rm face,1} = (x - 1/2\Delta x, y, z)$ and $\vec{x}_{\rm face,2} = (x + 1/2\Delta x, y, z)$. The value for the face at $x_1 = x - 1/2\Delta x$ is stored as $F^1[i, j, k]$. It's coordinates are stored at

$$\vec{x}_{\rm face,1} = (x_{\rm f}[i, j, k],\ y_{\rm c}[i, j, k],\ z_{\rm c}[i, j, k]), \tag{1}$$

where $x_{\rm f}$ is the face-centered 1D array of size $N_{x1} + 1$ while $y_{\rm c}$ and $z_{\rm c}$ are the cell-centered 1D arrays of size N_{x2} and N_{x3} respectively. The other face, which is located at $x_1 = x - 1/2\Delta x$, is stored in $F^1[i + 1, j, k]$ with coordinates

$$\vec{x}_{\rm face,2} = (x_f[i + 1, j, k], y_c[i, j, k], z_c[i, j, k]). \tag{2}$$

This means that F^1, F^2, and F^3 all have different sizes with an extra cell in the dimension corresponding to the components direction. F^1, for example, is of size $(N_{x1} + 1) \times N_{x2} \times N_{x3}$. `KHARMA` stores all three arrays in a single 4D array of size $(N_{x1} + 1) \times (N_{x2} + 1) \times (N_{x3} + 1)$ with dead cells for padding out each component. The active indices for this matrix are then,

$$F[1 : N_{x1} + 1,\ 1 : N_{x2},\ 1 : N_{x3}, 1], \tag{3}$$
$$F[1 : N_{x1},\ 1 : N_{x2} + 1,\ 1 : N_{x3}, 2], \tag{4}$$
$$F[1 : N_{x1},\ 1 : N_{x2},\ 1 : N_{x3} + 1, 3]. \tag{5}$$

1.1.2 Primitive and Conservative Variables

`KHARMA` utilizes cell-centered values for all primitive variables and conservative values $\rho_{\rm con}$, $u_{\rm con}$, $\mathbf{u}_{\rm con}$. The conservative B field variable, however, is stored as the face-centered representation $\mathbf{B}_{\rm con,f}$.

All conservative variables are stored as the desensitized version, meaning that the variable value is a density per unit area. For the B field, this means $B^{\rm d}_{\rm con,f} = \sqrt{-g}\mathcal{B}^{\rm d}$, where $\mathcal{B}$ is the raw field value and $\sqrt{-g}$ is the determinant of the metric.

1.2 Magnetic Field Divergence

Two relevant operations have the potential to create a divergence in the altered B fields. One is from transforming from one grid to another, $\mathcal{G}_0 \rightarrow \mathcal{G}_{\rm new}$. The other is switching representations from cell-centered to face-centered values. `KHARMA` evolves the magnetic field through a separate transport scheme that only utilizes face-centered values, so neither of these operations is utilized. However, because we want to ultimately resize our simulations, we will be utilizing both of these operations. We must do so carefully to avoid spurious monopoles causing many numerical errors and ultimately corrupting the simulation data.

2 METHODS

2.1 Resizing the Grid Resolution

We define the act of resizing the simulation grid as keeping the grid's domain identical, but changing the number of grid cells used to divide up that space. Here, we focus on the more difficult operation of upscaling the resolution, as downscaling will come free with the proper upscaling method.

Once we've initialized a new grid with an increased resolution, we must decide how to select values for the new grid cells that best represent the old grids value. The simplest method is Nearest Neighbor (NN) selection, where one picks each new grid cell's value to be that of the old grid cell closest to the new cells coordinates. This is what `KHARMA` currently implements. As we will see, however, NN results

© 2026 The Authors

in field less accurate to the original and creates a larger divergence when compared to the alternative method of creating a trilinear interpolation of the old grid. Nonetheless, `Juliarma` implements both methods.

There exist more complicated methods that upscale the grid without creating a divergence (Tóth–Roe, upscaling the field potential rather than the actual field, local iterative cleaning), however these come with their own separate limitations beyond our scope.

2.2 Cleaning the Resulting Divergence

Regardless of whether nearest neighbor or a trilinear interpolation is chosen, we now have a resulting upscaled field $\mathbf{B}$ in which $\nabla \cdot B \neq 0$. To fix this, we will create a new field $\delta\mathbf{B}$ to add to the original such that $\nabla \cdot (\mathbf{B} - \delta\mathbf{B}) = 0$. To find $\delta\mathbf{B}$, we solve for a potential Φ such that

$$\nabla \cdot \mathbf{B} = \nabla^2 \Phi. \tag{6}$$

The new field is then

$$\mathbf{B}' = \mathbf{B} - \nabla\mathbf{\Phi}. \tag{7}$$

2.2.1 *KHARMA's Discrete Operators*

To implement this solver in `Juliarma`, we must carefully define the discrete operators for the divergence and Laplacian in accordance with `KHARMA`'s conventions.

The Laplacian operator $\mathbf{L}$ is defined as,

$$\begin{aligned}(\mathbf{L}\Phi)_{i,k,j} &= \sum_d \frac{\Phi_{+\mathrm{d}} + 2\Phi_{i,j,k} + \Phi_{-d}}{(\Delta x_\mathrm{d})^2}, \\ &= \frac{\Phi[i+1,j,k] - 2\Phi[i,j,k] + \Phi[i-1,j,k]}{(\Delta x_1)^2} \\ &\quad + \frac{\Phi[i,j+1,k] - 2\Phi[i,j,k] + \Phi[i,j-1,k]}{(\Delta x_2)^2} \\ &\quad + \frac{\Phi[i,j,k+1] - 2\Phi[i,j,k] + \Phi[i,j,k-1]}{(\Delta x_1)^2}.\end{aligned} \tag{8}$$

The gradient operator $\mathbf{G}$ is,

$$(\mathbf{G}\Phi)^1_{i,k,j} = \frac{\Phi[i,j,k] - \Phi[i-1,j,k]}{\Delta x_1}, \tag{9}$$

with the same logic following for the other two components. As defined here, the resulting components will have a staggered shape. That is, $\mathrm{size}(\mathbf{G}\Phi)^1 = (N_{x1}+1) \times N_{x2} \times N_{x3}$.

The divergence operator $\mathbf{D}$ is,

$$(\mathbf{DB})_{i,k,j} = \sum_d \frac{B_{+\mathrm{d}} + B_{i,j,k}}{\Delta x_\mathrm{d}}, \tag{10}$$

$$= \frac{B[i+1,j,k] + B[i,j,k]}{\Delta x_1} \tag{11}$$

$$+ \frac{B[i,j+1,k] + B[i,j,k]}{\Delta x_2}, \tag{12}$$

$$+ \frac{B[i,j,k+1] + B[i,j,k]}{\Delta x_3}. \tag{13}$$

2.2.2 *Boundary Conditions*

The edge cases for the previously created matrices introduce non-trivial choices for padding that depend upon boundary conditions. In general, we can pad a matrix A by asserting that it is zero on the boundary (Dirichlet), has a zero normal derivative on the boundary (Neumann), or is periodic on the boundary. The ghost cell value Ω_g for a given boundary cell Ω_b is as follows.

- Dirichlet: $\Omega_\mathrm{g} = \Omega_\mathrm{b}$.
- Neumann: $\Omega_\mathrm{g} = \Omega_\mathrm{b}$.
- Periodic: Ω_g = cell on opposite boundary of Ω_b.

There is a fourth boundary condition highly relevant to the coordinate singularity located at the poles for spherical coordinates. This is the transmitting polar boundary condition. In this case, the ghost cells at the north pole $\theta = 0$ are copied from the values at the opposite side of that pole shifted in the ϕ direction by π. For a scalar field A, the value chosen for the ghost cell opposite of the $\theta = 0$ boundary is $A[r, \theta_1, \phi + \pi]$, where θ_1 indicates the first active cell. For a vector field, crossing the pole means changing the unit vector associated with that cell. That is, a vector pointing towards the pole on one side will need its sign flipped when represented in an array at a ϕ coordinate shifted by π to still be pointing in the same direction. This means that a vector field with values F_r, F_θ, F_ϕ will have its value as $F_r, -F_\theta, -F_\phi$ when stored as a ghost cell.

The boundary conditions for Φ must be chosen so that the resulting $\mathbf{B}$ matches the grids original boundary conditions. Futher, for Equation 6 to be solvable, there must be no constants in the null space so that the corresponding solving matrix is invertible. This adds extra restrictions onto our boundary conditions. [WIP]

2.3 Primitive to Conservative Conversion

The remaining resized primitive variables are much simpler to obtain. Because there no extra restrictions on them, a simple trilinear interpolation or nearest neighbor selection is used to populate the new matrices for ρ_prim, u_prim, and $\mathbf{u}_\mathrm{prim}$.

The `KHARMA` `rhdf` files also store the conservative variables. These cannot be as simply upscaled for our resized file because there is a tight relation between the conservative and primitive variables that would be broken if both are interpolated. We must perform the variable conversions from our new primitive variables ourselves with the same conventions `KHARMA` uses.

2.3.1 *Conversion Equations*

We start with the primitive velocity vector and calculate the following,

$$q^2 = g_{i,j} u^i_\mathrm{prim} u^j_\mathrm{prim}, \tag{14}$$

$$\Gamma = \sqrt{1 + q^2}. \tag{15}$$

The lapse function (rate of proper time evolves relative to coordinate time) is $\alpha = 1/\sqrt{-g^{00}}$. We next calculate the contravariant four-velocity.

$$\tilde{u}^0 = \frac{\Gamma}{\alpha}, \quad \tilde{u}^i = u^i_\mathrm{prim} - \alpha\Gamma g^{0i} \tag{16}$$

We then contract these values with the metric to lower the indices,

$$\tilde{u}_\mu = g_{\mu\nu}\tilde{u}^\nu. \tag{17}$$

We now have the `KHARMA` consistent $\mathbf{u}_\mathrm{cons} = \tilde{u}_\mu$

To calculate the stress-energy tensor, we'll need the cell-centered primitive B field $\mathbf{B}_\mathrm{prim,c}$. This variable is not stored in the `KHARMA` `rhdf` file. To obtain it, we perform the same conversion operation `KHARMA` does and take the average of the two cell faces normal to a given component's direction,

$$\mathbf{B}_\mathrm{prim,c}[i,j,k,1] = \frac{1}{2}\left(\frac{\mathbf{B}_\mathrm{cons,f}[i,j,k,1]}{\mathrm{gdet}[i,j,k]} + \frac{\mathbf{B}_\mathrm{cons,f}[i+1,j,k,1]}{\mathrm{gdet}[i+1,j,k]}\right). \tag{18}$$

The same logic is used for the other 2 directions as well. Here, gdet = $\sqrt{-g}$ and undoes the desensitization of the conservative variables

We next construct the invariant magnetic field intensity b^2,

$$b^0 = B^i u_i, \quad b^i = \frac{B^i + b^0 \tilde{u}^i}{\tilde{u}^0}, \tag{19}$$

$$b^2 = b^\mu b_\mu, \tag{20}$$

then use this quantity to calculate the stress energy tenser,

$$\eta = \rho_{\text{prim}} + u_{\text{gas}} + p + b^2, \tag{21}$$

$$p_{\text{tot}} = p + \frac{b^2}{2}, \tag{22}$$

$$T^0{}_\mu = \eta\, \tilde{u}^0 \tilde{\mathbf{u}}_\mu + p_{\text{tot}} \delta^0{}_\mu - b^0 b_\mu. \tag{23}$$

where pressure p is derived from our simulation's gamma-law equation of state,

$$p = (\gamma_{\text{ad}} - 1) u_{\text{prim}}. \tag{24}$$

We can now calculate the conservative variables,

$$\rho_{\text{cons}} = \sqrt{-g}\, \rho_{\text{prim}} \tilde{u}^0, \tag{25}$$

$$u_{\text{cons}} = \sqrt{-g}\, T^0{}_0 + \rho_{\text{cons}}, \tag{26}$$

$$\mathbf{u}_{\text{cons}} = \sqrt{-g}\, T^0{}_i. \tag{27}$$

which again are densitized.

2.3.2 Evaluating the Metric

The above calculations are all done on the new grid $\mathcal{G}_{\text{new}}$. As such, all variables used must have the appropriate size. We've already obtained $\mathbf{B}_{\text{cons,f}}$ and the primitive variables via trilinear interpolation (and a cleanup for the magnetic field). This still leaves us without the matrices for the covariant and contravariant metric, and for gdet. These can be interpolated as well, but this would add unnecessary inaccuracies. The `phdf` files contain all that is needed to properly evaluate the metric at on our finer coordinate grid.

We do this through `PyHarm` in the `pyharmConverter.py` script. There, we use `pyharm.grid.Grid` to create a new grid with our new resolution from the original dump file. `Pyharm` then uses the parameter values stored in the dump file to recompute the metric at the new coordinate locations. These new metric values are stored in a temporary file to be used and eventually deleted later in `Juliarma`'s pipeline.

2.4 Restrictions

All the calculations used in `Juliarma` assume a uniform grid. This means we cannot resize any simulation that uses static or adaptive mesh refinement. Also, the code has no `MPI` capabilities. This means that the program must be run on a single node or computer. Fortunately, `KHARMA`'s dump files are rather small even for large simulations, so it's unlikely that resizing will ever be computationally expensive enough to require multiple nodes.

3 RESULTS AND DISCUSSIONS

4 CONCLUSIONS

ACKNOWLEDGEMENTS

REFERENCES

This paper has been typeset from a TEX/LATEX file prepared by the author.